

Wydział Informatyki Politechniki Białostockiej

Systemy wbudowane

Tytuł: Przełączniki SW[9:0], przyciski KEY[3:0], Diody LEDR[9:0] i
multipleksery (cz. 2)

Wykonujący: Łukasz Modzelewski, Mariusz Rostkowski

PS 1

Zespół 4

Praca laboratoryjna 3

Studia dzienne

Informatyka

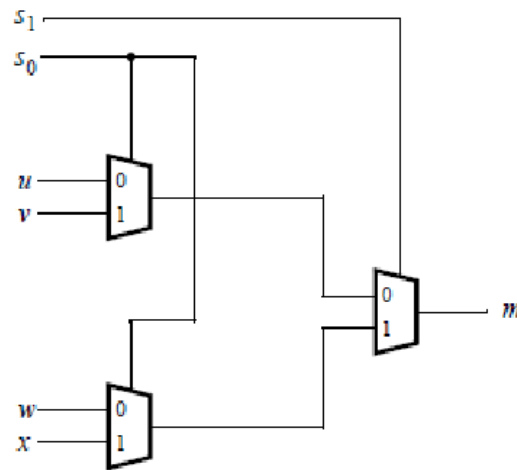
Semestr VI

Prowadzący: prof. dr hab. inż. Valery Salaouyou

Data: 17.03.2022

Zadanie 1

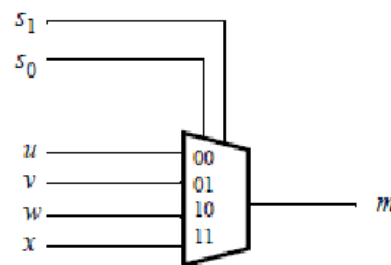
Utwórz i zaimplementuj projekt **mux_4_1_1_bit** multipleksera "4-1"



a) Circuit

s_1	s_0	m
0	0	u
0	1	v
1	0	w
1	1	x

b) Truth table



c) Symbol

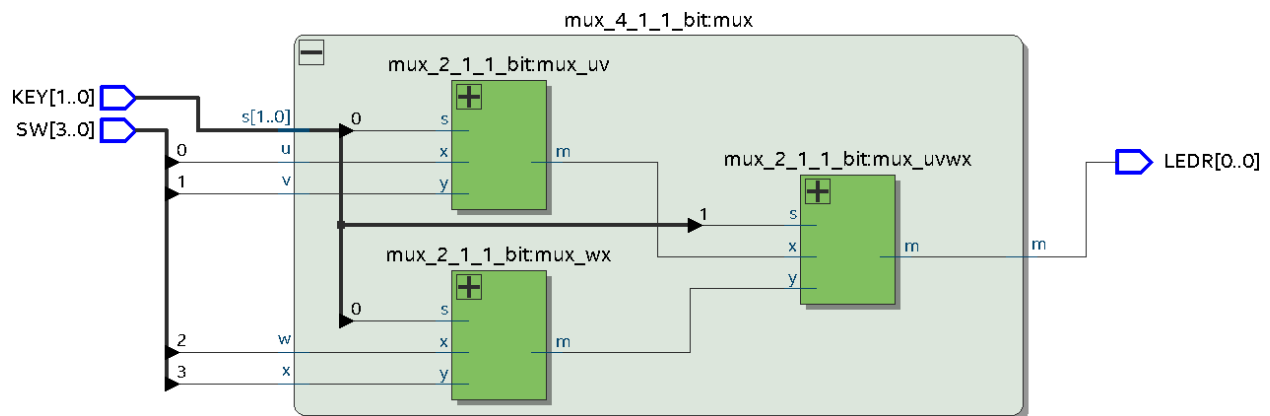
z następującym przyporządkowaniem pinów:

Wyprowadzenia	Piny
u	SW0
v	SW1
w	SW2
x	SW3
S1	KEY1
S0	KEY0
m	LED0

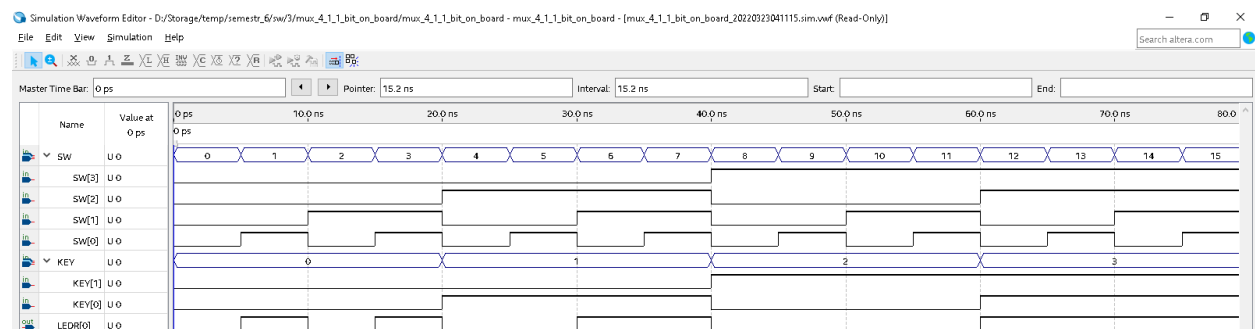
Kod w Verilog

```
module mux_4_1_1_bit_on_board(  
    input [3:0] SW,  
    input [1:0] KEY,  
    output [0:0] LEDR);  
  
    mux_4_1_1_bit mux(SW[0], SW[1], SW[2], SW[3], KEY[1:0], LEDR[0]);  
  
endmodule  
  
module mux_4_1_1_bit(  
    input u, v, w, x,  
    input [1:0] s,  
    output m);  
  
    wire m_uv, m_wx;  
    mux_2_1_1_bit mux_uv(u, v, s[0], m_uv);  
    mux_2_1_1_bit mux_wx(w, x, s[0], m_wx);  
    mux_2_1_1_bit mux_uvwx(m_uv, m_wx, s[1], m);  
  
endmodule  
  
module mux_2_1_1_bit(  
    input x, y,  
    input s,  
    output m);  
  
    assign m = (~s & x) | (s & y);  
  
endmodule
```

Wyniki syntezy



Wyniki symulacji funkcjonalnej

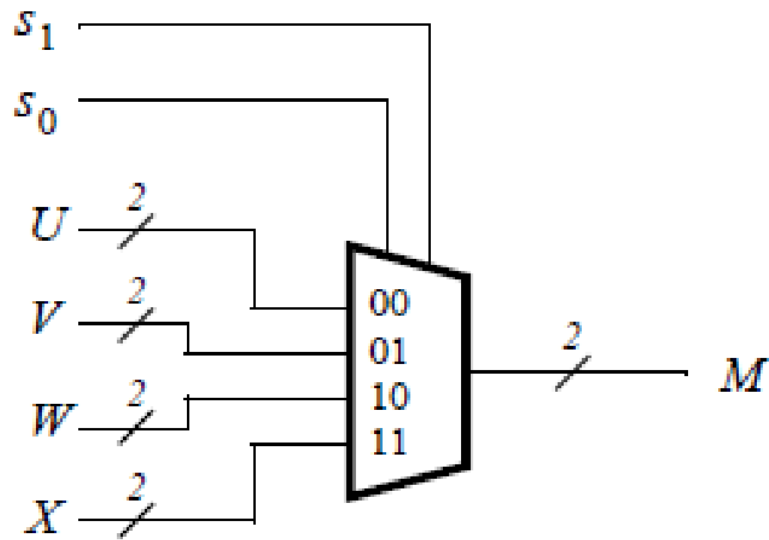


Działanie na płycie



Zadanie 2

Utwórz i zaimplementuj projekt **mux_4_1_2_bits** multiplexera 4 x 1 o szerokości 2 bitów



z następującymi przyporządkowaniami pinów:

Wyprowadzenia	Piny
U[1:0]	SW[1:0]
V[1:0]	SW[3:2]
W[1:0]	SW[5:4]
X[1:0]	SW[7:6]
S[1:0]	KEY[1:0]
M[1:0]	LEDR[1:0]

Kod w Verilog

```

module mux_4_1_2_bits_on_board(
    input [7:0] SW,
    input [1:0] KEY,
    output [1:0] LEDR);

    mux_4_1_2_bits mux(SW[1:0], SW[3:2], SW[5:4], SW[7:6], KEY[1:0], LEDR[1:0]);
endmodule

module mux_4_1_2_bits(
    input [1:0] u, v, w, x,
    input [1:0] s,
    output [1:0] m);

    mux_4_1_1_bit mux0(u[0], v[0], w[0], x[0], s[1:0], m[0]);
    mux_4_1_1_bit mux1(u[1], v[1], w[1], x[1], s[1:0], m[1]);
endmodule

module mux_4_1_1_bit(
    input u, v, w, x,
    input [1:0] s,
    output m);

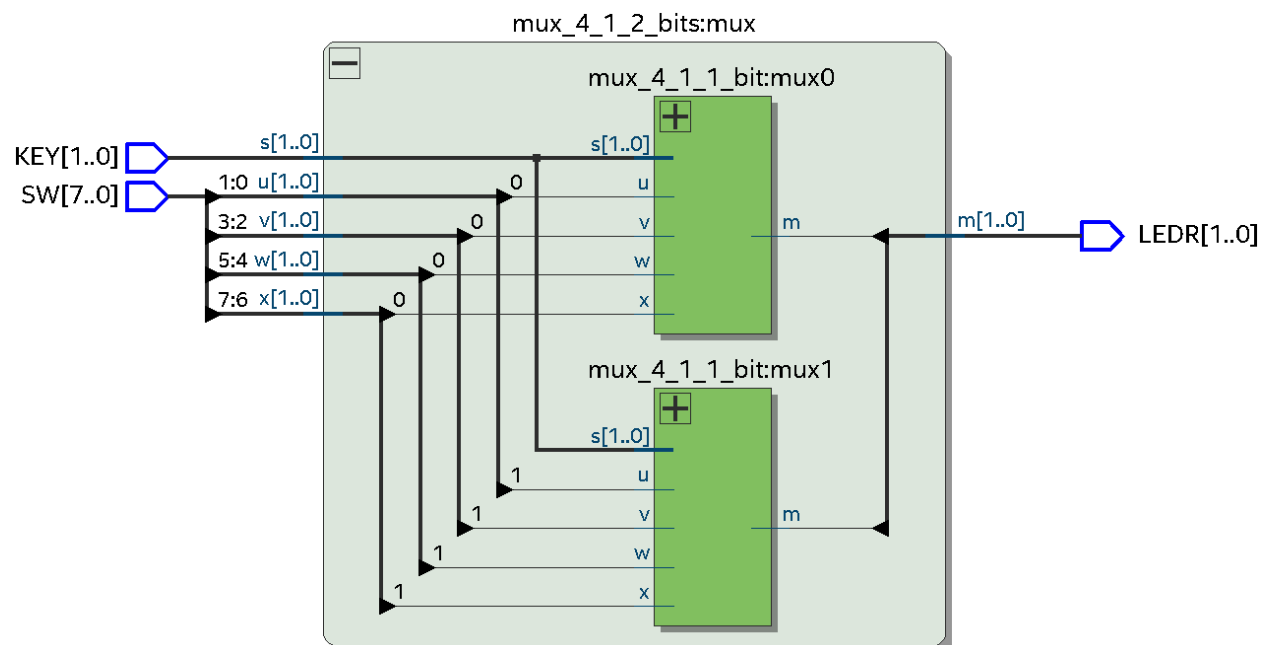
    wire m_uv, m_wx;
    mux_2_1_1_bit mux_uv(u, v, s[0], m_uv);
    mux_2_1_1_bit mux_wx(w, x, s[0], m_wx);
    mux_2_1_1_bit mux_uvwx(m_uv, m_wx, s[1], m);
endmodule

module mux_2_1_1_bit(
    input x, y,
    input s,
    output m);

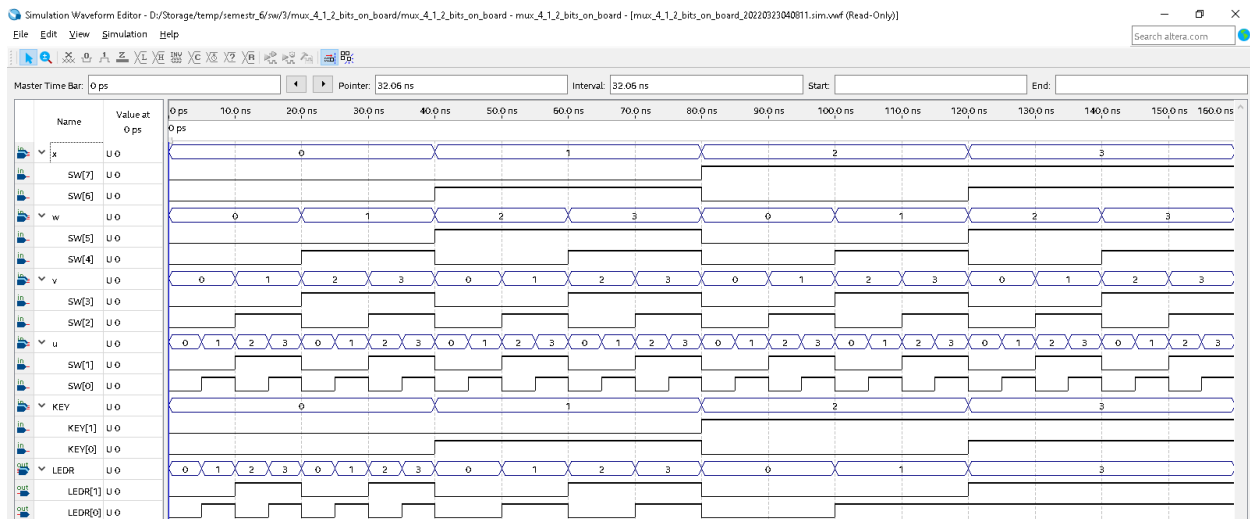
    assign m = (~s & x) | (s & y);
endmodule

```

Wyniki syntezy

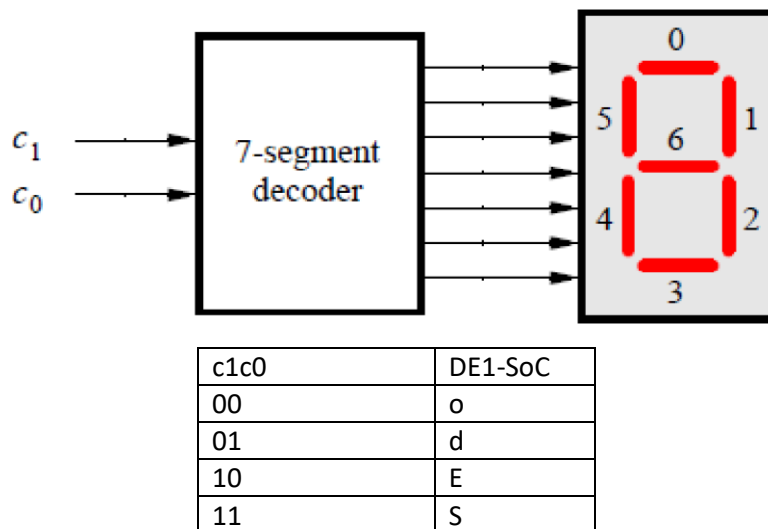


Wyniki symulacji funkcjonalnej



Zadanie 3

Utwórz i wykonaj symulację projektu decoder_7_seg, który przekształca 2-bitowy kod znaków w sygnały sterujące wyświetlaczem 7-segmentowym.



Zaimplementuj na tablicy i przetestuj projekt za pomocą następującego przydziału pinów:

Wyprowadzenia	Piny
C[1:0]	SW[1:0]
h[0:6]	HEX0[0:6]

Wybraliśmy znaki o, d, E, S z nazwiska Modzelewski.

Stworzyliśmy tabelę prawdy, która dla kolejnych wartości wejść dekodera c[1:0] opisuje, czy segmenty h[6:0] wyświetlacza są zapalone (0), czy zgaszone (1).

c[1:0] (dec)	c1	c0	h6	h5	h4	h3	h2	h1	h0	h[6:0] (dec)	znak
0	0	0	0	1	0	0	0	1	1	35	o
1	0	1	0	1	0	0	0	0	1	33	d
2	1	0	0	0	0	0	1	1	0	6	E
3	1	1	0	0	1	0	0	1	0	18	S

Metodą map Karnaugh'a wyznaczyliśmy funkcje wyjść h[6:0].

h6, h3 = 0

c1 \ c0	0	1	c1 \ c0	0	1	c1 \ c0	0	1	c1 \ c0	0	1
0	1	1	0	0	0	0	0	0	0	1	0
1	0	0	1	0	1	1	1	0	1	1	1
h5, h0 = c1'			h4 = c1*c0			h2 = c1*c0'			h1 = c1 + c0'		

Kod w Verilog

```
module decoder_7_seg_on_board(
    input [1:0] Sw,
    output [6:0] HEX0);

    decoder_7_seg decoder(Sw[1:0], HEX0[6:0]);

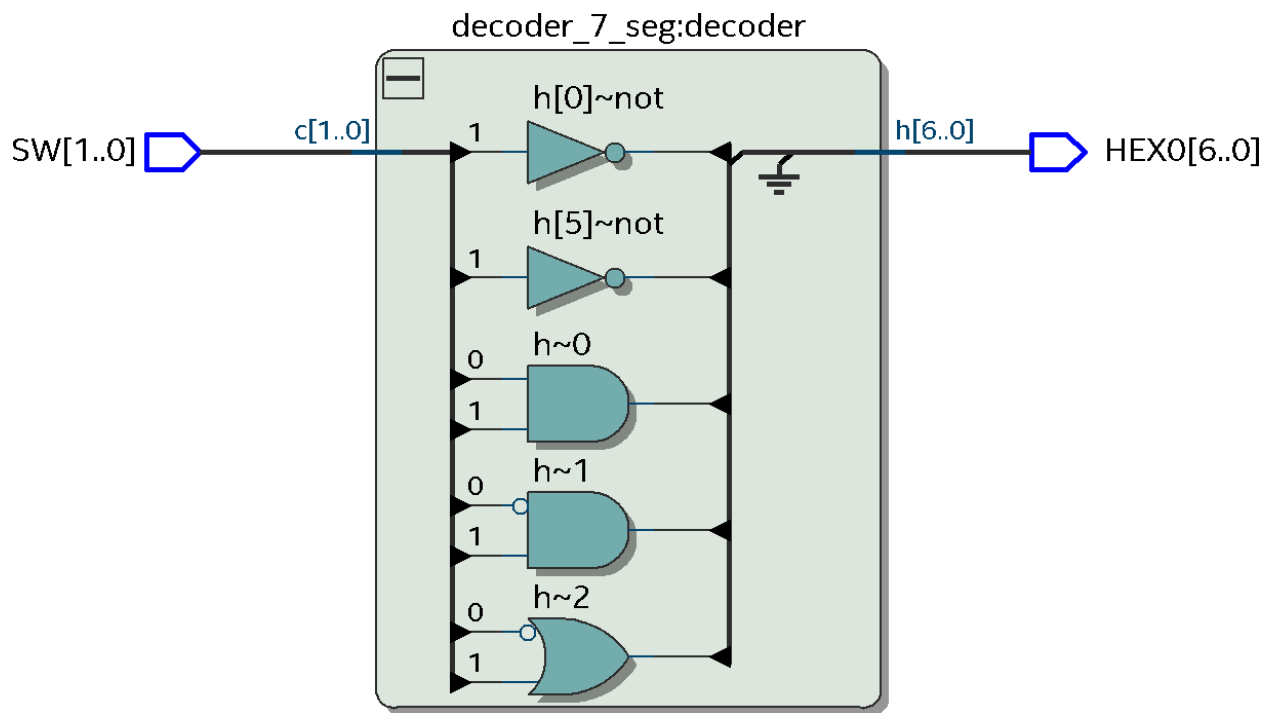
endmodule

module decoder_7_seg(
    input [1:0] c,
    output [6:0] h);

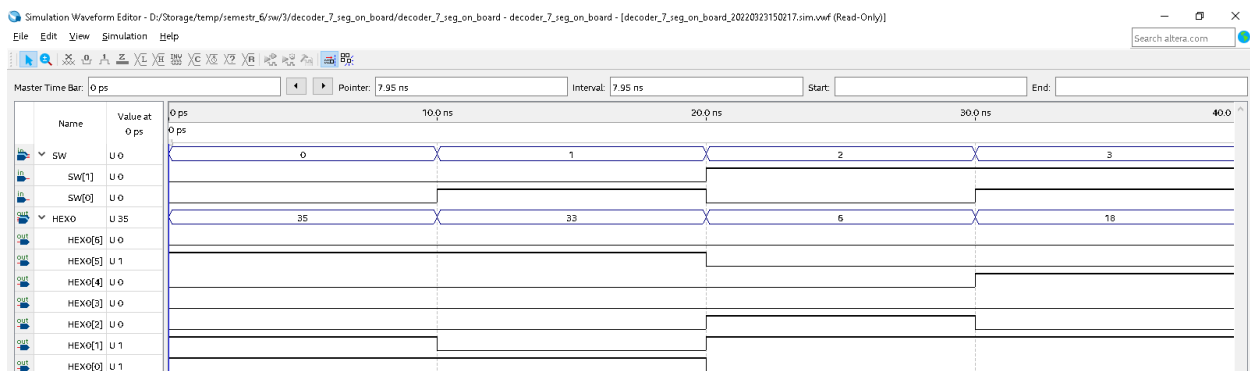
    assign {h[6], h[3]} = {2{ 1'b0 }};
    assign {h[5], h[0]} = {2{ ~c[1] }};
    assign h[4] = c[1] & c[0];
    assign h[2] = c[1] & (~c[0]);
    assign h[1] = c[1] | (~c[0]);

endmodule
```

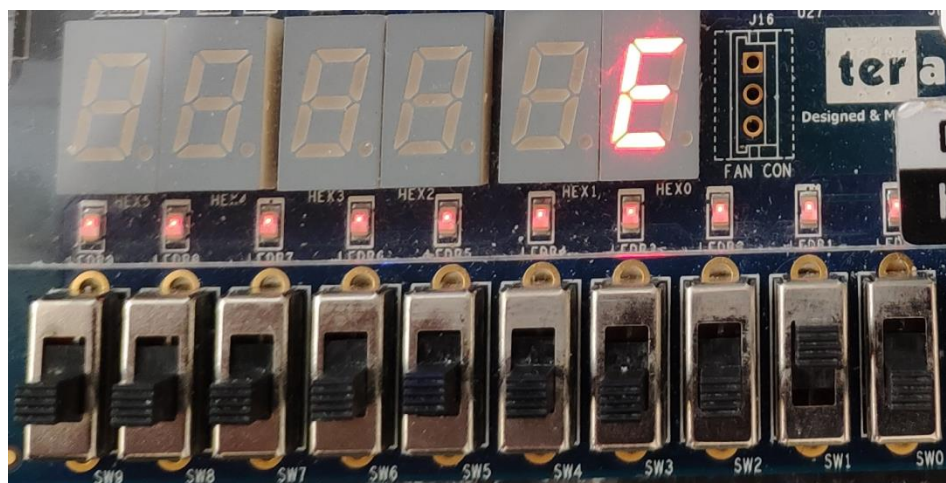
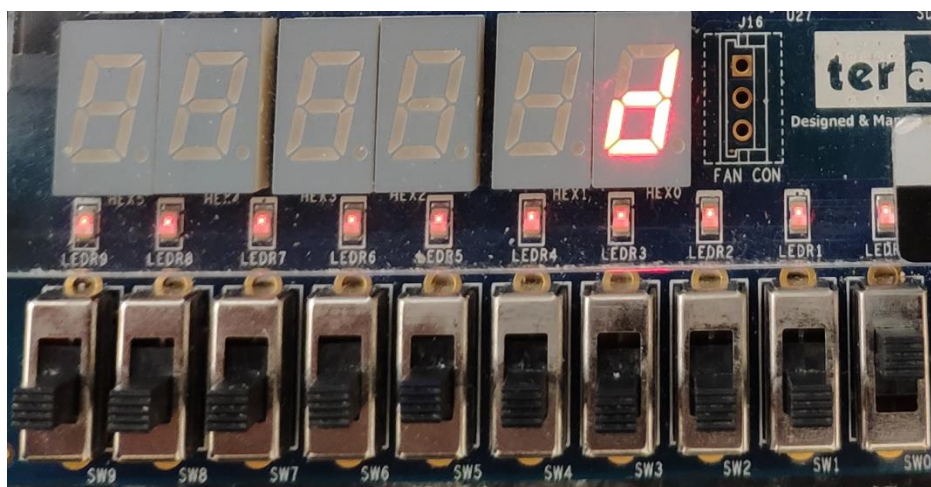
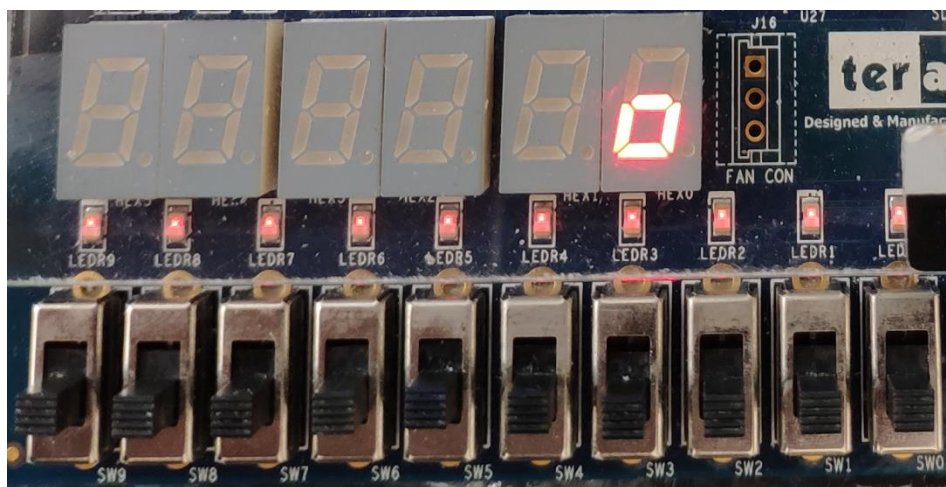
Wyniki syntezy

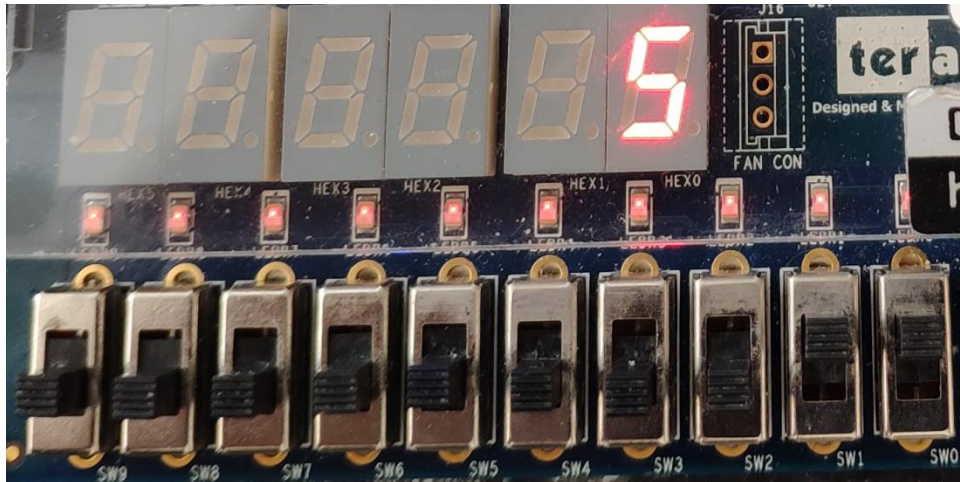


Wyniki symulacji funkcjonalnej



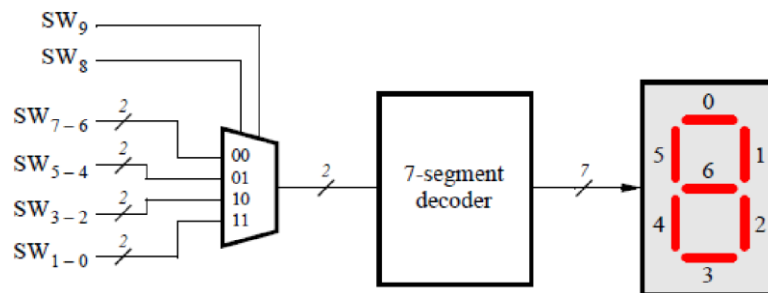
Działanie na płycie





Zadanie 4

Utwórz i zaimplementuj projekt choose_from_4_symbols, który pozwala wybrać i wyświetlić na HEX0 jeden z czterech symboli:



Uwaga: wartości przełączników SW[9:0] mają być dodatkowo wyświetlane na diodach LED LEDR[9:0].

Kod w Verilog

```
module choose_4_from_symbols_on_board(
    input  [9:0] SW,
    output [9:0] LEDR,
    output [6:0] HEX0);

    assign LEDR = SW;
    wire [1:0] c;
    mux_4_1_2_bits mux(SW[7:6], SW[5:4], SW[3:2], SW[1:0], SW[9:8], c[1:0]);
    decoder_7_seg decoder(c[1:0], HEX0[6:0]);

endmodule

module mux_4_1_2_bits(
    input  [1:0] u, v, w, x,
    input  [1:0] s,
    output [1:0] m);

    mux_4_1_1_bit mux0(u[0], v[0], w[0], x[0], s[1:0], m[0]);
    mux_4_1_1_bit mux1(u[1], v[1], w[1], x[1], s[1:0], m[1]);

endmodule
```

```

module mux_4_1_1_bit(
    input u, v, w, x,
    input [1:0] s,
    output m);

    wire m_uv, m_wx;
    mux_2_1_1_bit mux_uv(u, v, s[0], m_uv);
    mux_2_1_1_bit mux_wx(w, x, s[0], m_wx);
    mux_2_1_1_bit mux_uvwx(m_uv, m_wx, s[1], m);

endmodule

module mux_2_1_1_bit(
    input x, y,
    input s,
    output m);

    assign m = (~s & x) | (s & y);

endmodule

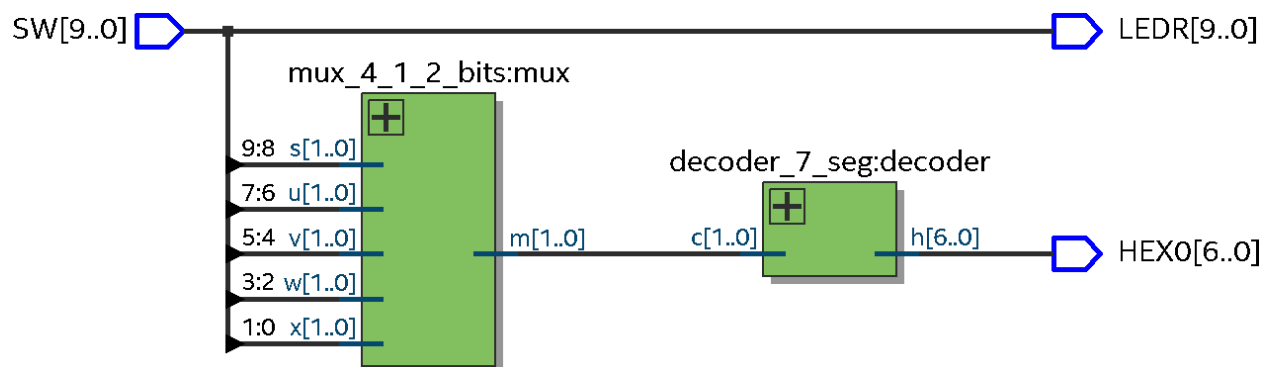
module decoder_7_seg(
    input [1:0] c,
    output [6:0] h);

    assign {h[6], h[3]} = {2{ 1'b0 }};
    assign {h[5], h[0]} = {2{ ~c[1] }};
    assign h[4] = c[1] & c[0];
    assign h[2] = c[1] & (~c[0]);
    assign h[1] = c[1] | (~c[0]);

endmodule

```

Wyniki syntezy



Wyniki symulacji funkcjonalnej

Wejścia informacyjne multipleksera:

- u = 1
- v = 0
- w = 3
- x = 2

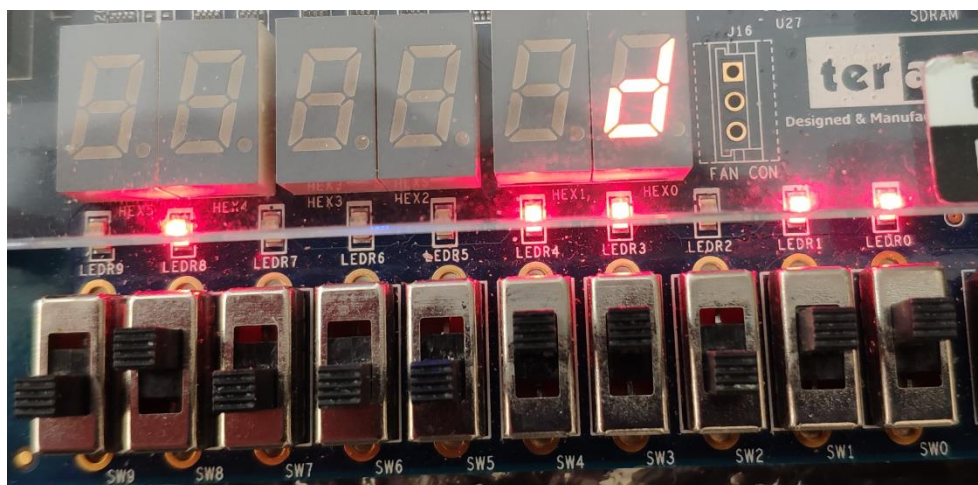
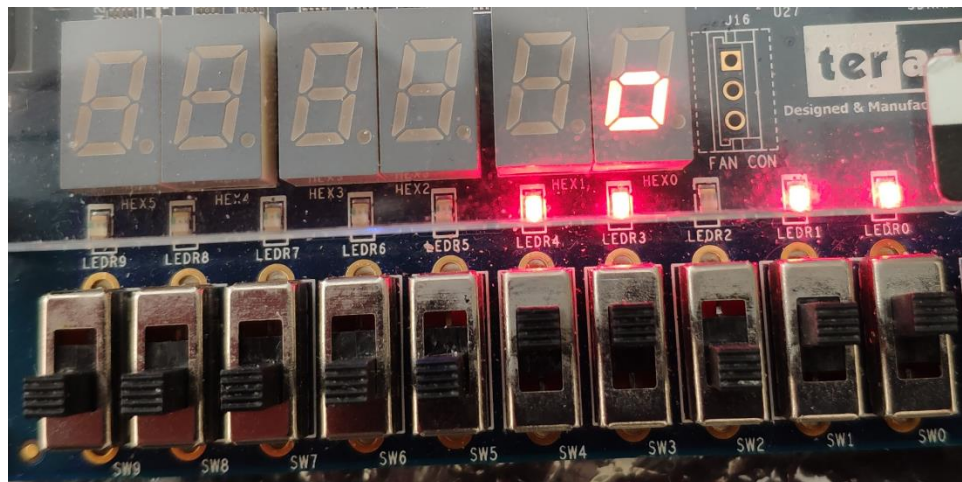
Wejścia wybierające multipleksera:

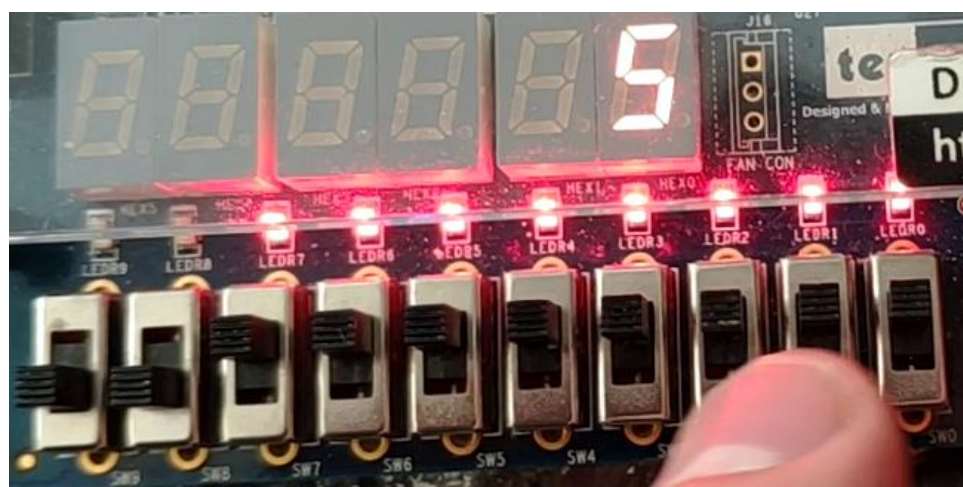
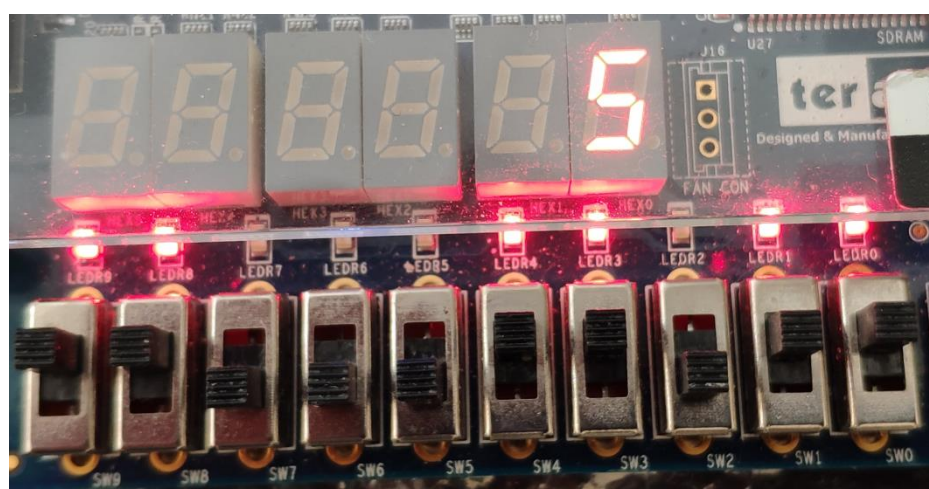
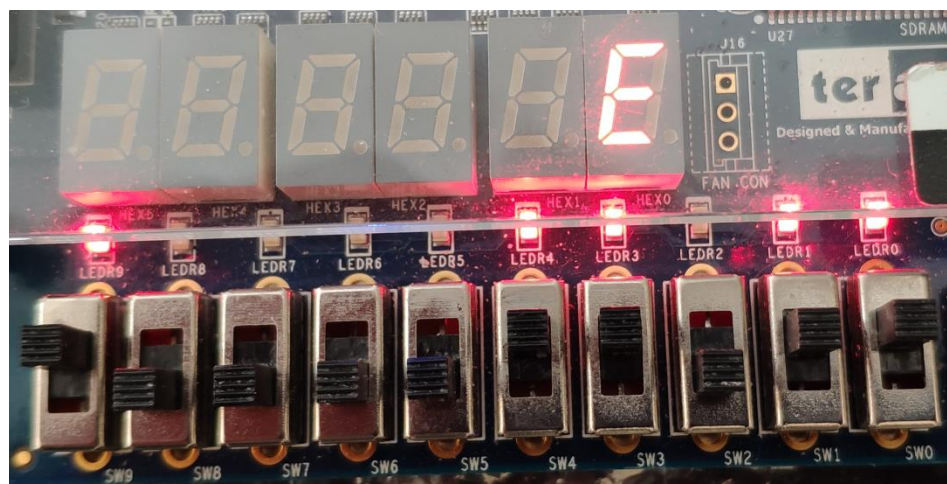
- 0 -> wybrane u = 1 -> na wyjściu dekodera 33, czyli na HEX0 wyświetlony znak d

- 1 -> wybrane v = 0 -> na wyjściu dekodera 35, czyli na HEX0 wyświetlony znak o
- 2 -> wybrane w = 3 -> na wyjściu dekodera 18, czyli na HEX0 wyświetlony znak S
- 3 -> wybrane x = 2 -> na wyjściu dekodera 6, czyli na HEX0 wyświetlony znak E



Działanie na płycie





Wnioski

Wszystkie zadania zostały zrealizowane.